

In this chapter, we give the reader an initial knowledge base on all the theoretical concepts, and all the components that had been utilized in the development of this research.

In the first section we will provide a description of CP itself, starting from a general definition, delving deeper in the description of CP solvers, and then we are gonna provide a brief description of MiniZinc, a CP modelling language, in which all the tested problems are coded.

In the second section we will provide an high level description of what large language models (LLMs) are and how they work, starting from the implementation of inference and decoding, passing to prompting techniques and prompt engineering, finally explaining the use of Application Programming Interfaces (APIs) as a communication mean.

0.1 Constraint Programming

CP is a paradigm for declarative description and effective solving of large, particularly combinatorial, problems especially in areas of planning and scheduling [?].

A constraint can be thought of intuitively as a formalization of dependencies in physical worlds and their mathematical abstractions. A constraint is a logical relation among several unknowns (or variables), each taking a value in a given domain. The constraint thus restricts the possible values that variables can take; it represents partial information about the variables of interest. Constraints can also be heterogeneous, so they can bind unknowns from different domains, for example a length (number) with a word (string). The important feature of constraints is their declarative manner, i.e., they specify what relationship must hold without specifying a computational procedure to enforce that relationship [?].

Constraints arise naturally in most areas of human endeavor. They are the natural medium of expression for formalizing regularities that underlie the computational and (natural or designed) physical worlds and their mathematical abstractions.

CP is the study of computational systems based on constraints. The idea of CP is to solve problems by stating constraints (requirements) about the problem area and, consequently, finding a solution satisfying all the constraints.

0.1.1 Solvers

Throughout this thesis, the term solver denotes a system that, given a formal problem specification, computes assignments satisfying a set of constraints and, when required, optimizes an objective function.

A “Constraint Satisfaction Problem” (CSP) can be formalized as a tuple

$$\langle X, D, C \rangle,$$

where X is a finite set of decision variables, $D(x)$ denotes the domain associated with each variable $x \in X$, and C is a set of constraints defined over subsets of X . A solution is an

assignment

$$s : X \rightarrow \bigcup_{x \in X} D(x)$$

such that $s(x) \in D(x)$ for all $x \in X$ and all constraints in C are satisfied [?].

The objective in a CSP is to determine whether at least one feasible assignment exists, or to enumerate all feasible assignments.

A “Constraint Optimization Problem” (COP) extends the CSP by introducing an objective function. Formally, a COP can be defined as

$$\langle X, D, C, f \rangle,$$

where $f : S \rightarrow \mathbb{R}$ is an objective function defined over the set S of feasible assignments. The goal is to compute an optimal solution

$$s^* = \arg \min_{s \in S} f(s) \quad \text{or} \quad s^* = \arg \max_{s \in S} f(s),$$

depending on whether the problem is a minimization or maximization task.

Different solver families address CSPs and COPs using distinct mathematical abstractions and algorithmic techniques.

Constraint Programming (CP) solvers

Constraint Programming solvers operate directly on variables with finite or structured domains and heterogeneous constraints. Their fundamental mechanisms are constraint propagation, which removes inconsistent values from variable domains, and systematic search, typically implemented via backtracking guided by variable- and value-ordering heuristics. CP solvers natively support high-level global constraints equipped with specialized filtering algorithms that exploit combinatorial structure [?]. A representative example is Gecode [?].

Linear and Mixed-Integer Linear Programming solvers

Linear Programming (LP) and Mixed-Integer Linear Programming (MILP) solvers address optimization problems in which the objective function and the constraints are linear. A standard LP formulation is

$$\min \{c^\top x \mid Ax \leq b, x \in \mathbb{R}^n\},$$

where $x \in \mathbb{R}^n$ is a vector of continuous decision variables, $c \in \mathbb{R}^n$ defines the linear objective function, and $Ax \leq b$ represents a system of linear inequalities.

LP solvers assume continuous domains and rely on algorithms such as the simplex method and interior-point methods [?]. They do not natively support discrete decision variables or high-level combinatorial constraints; such constructs must be reformulated as linear expressions over continuous variables.

Mixed-Integer Linear Programming extends this framework by restricting some or all variables to integer domains, i.e., $x_i \in \mathbb{Z}$ for selected indices. MILP solvers typically combine LP relaxations with tree-based search procedures such as branch-and-bound and branch-and-cut [?]. This hybrid approach enables the treatment of discrete decisions within a linear modeling framework, although global or non-linear constraints must generally be encoded through auxiliary variables and additional linear constraints. Examples of solvers in this family include HiGHS [?] and Gurobi [?].

Portfolio Solvers

Solving combinatorial search problems is hard, and there exist nowadays plenty of techniques and constraint solvers for performing this task. It has become clear that different solvers are better when solving different problem instances, even within the same problem class. It has also been shown that a single, arbitrarily efficient solver can be significantly outperformed by using a portfolio of possibly on-average slower solvers [?].

Algorithm portfolios [?] can be seen as instances of the more general Algorithm Selection problem [?] where, as reported in [?], the algorithm selection is performed case-by-case for each problem to solve. Within the context of constraint solving, a portfolio approach enables to combine a number $m > 1$ of different constituent solvers $s_1, \dots, s_m$ in order to create a globally better constraint solver, dubbed a portfolio solver. When a new, unseen problem p comes, the portfolio solver tries to predict the best constituent solver(s) $s_{i_1}, \dots, s_{i_k}$ (with $1 \leq i_j \leq m$ for $j = 1, \dots, k$) for solving p and then runs them on p . Properly selecting and scheduling the solvers is a crucial step for the performance of a portfolio solver, and it is usually performed by exploiting Machine Learning techniques based on features extracted from the problem p to solve.

In the course of this research thesis, starting from the idea behind portfolio solvers, we will try to evaluate the performances of a pre-trained LLM to understand if this could be a viable solution to the algorithm selection problem [?], avoiding the cost of training for portfolio solvers, following the new paradigm proposed in [?].

0.1.2 MiniZinc

MiniZinc is a simple but expressive CP modelling language which is suitable for modelling problems for a range of solvers and a reasonable compromise between many design possibilities.

It was born from the necessity of a standard modelling language for constraint programming problems, it is also making solver benchmarking simpler [?].

For these reasons all of the problems employed for testing in the course of this research paper were written using MiniZinc.

A MiniZinc Example

A MiniZinc problem specification has two parts: (a) the model, which describes the structure of a class of problems; and (b) the data, which specifies one particular problem within this class. The pairing of a model with a particular data set is a model instance (sometimes abbreviated to instance).

The model and data may be in separate files. Data files can only contain assignments to parameters declared in the model. A user specifies data files on the command line, rather than naming them in the model file, so that the model file is not tied to any particular data file [?].

Each MiniZinc model is a sequence of items, which may appear in any order. Consider the MiniZinc model and example data for a restricted job shop scheduling problem in Figure 1 and Figure 2.

Line 0 is a comment, introduced by the “%” character.

Lines 1-5 are “variable declaration items”. Line 1 declares `size` to be an integer parameter, i.e. a variable that is fixed in the model. Line 20 (in the data file) is an “assignment item” that defines the value of `size` for this instance. Variable declaration items can include assignments, as in line 3. Line 4 declares `s` to be a 2D array of “decision variables”. Line 5 is an integer variable with a restricted range. Decision variables are distinguished by the `var` prefix.

Lines 7-8 show a user-defined “predicate item”, `no_overlap`, which constrains two tasks given by start time and duration so that they do not overlap in time.

Lines 10-17 show a “constraint item”. It uses the built-in `forall` to loop over each job, and ensure that: (line 12) the tasks are in order; (line 13) they finish before end; and (lines 14-16) that no two tasks in the same column overlap in time. Multiple constraint items are allowed, they are implicitly conjoined.

Line 19 shows a `solve` item. Every model must include exactly one solve item. Here we are interested in minimizing the end time. We can also maximize a variable or just look for any solution (`solve satisfy`).

There is one kind of MiniZinc item not shown by this example: “include items”. They facilitate the creation of multi-file models and the use of library files [?].

MiniZinc Challenge

Comparing constraint programming systems is fraught with difficulty. The reason is that there are so many components to a modern CP system, only some of which are implemented by some systems. To claim that one CP system is “better” than another is a bold claim, since there is almost certainly some problem for which the “worse” system allows a stronger model, or a better search, and performs better.

MiniZinc makes a good attempt to handle the most obvious obstacle: there are hundreds of potential global constraints, most handled by few or no systems. A standard input language, gives us the capability to compare different solvers. Hence, every year since 2008 the MiniZinc

```

0    % (square) job shop scheduling in MiniZinc
1    int: size;                                % size of problem
2    array [1..size,1..size] of int: d;        % task durations
3    int: total = sum(i,j in 1..size) (d[i,j]); % total duration
4    array [1..size,1..size] of var 0..total: s; % start times
5    var 0..total: end;                        % total end time
6
7    predicate no_overlap(var int:s1, int:d1, var int:s2, int:d2) =
8        s1 + d1 <= s2 /\ s2 + d2 <= s1;
9
10   constraint
11       forall(i in 1..size) (
12           forall(j in 1..size-1) (s[i,j] + d[i,j] <= s[i,j+1]) /\
13               s[i,size] + d[i,size] <= end /\
14               forall(j,k in 1..size where j < k) (
15                   no_overlap(s[j,i], d[j,i], s[k,i], d[k,i])
16               )
17       );
18
19   solve minimize end;

```

Figure 1: MiniZinc model (jobshop.mzn) for the job shop problem (the portrayed example was taken from [?]).

```

20    size = 2;
21    d = [ 2,5,
22          3,4 ];

```

Figure 2: MiniZinc data (jobshop2x2.dzn) for the job shop problem (the portrayed example was taken from [?]).

Challenge has been comparing different solvers that support MiniZinc [?].

The aim of the challenge is to compare various constraint solving technology on the same problems sets. The focus is on finite domain propagation solvers. An auxiliary aim is to build up a library of interesting problem models, which can be used to compare solvers and solving technologies.

Challenge participants provide a FlatZinc or MiniZinc solver and global constraint definitions specialized for their solver. Each solver is run on 100 MiniZinc model instances. For FlatZinc solvers, the MiniZinc compiler runs on the MiniZinc model and instance using the provided global constraint definitions to create a FlatZinc file. The resultant FlatZinc file is then given as input to the provided FlatZinc solver. For MiniZinc solvers, the MiniZinc model and data are input to the provided solver.

Points are awarded for solving problems, speed of solution, and goodness of solutions (for optimization problems) [?], however, the MiniZinc challenge scoring system is not actually utilized in this specific research, since it is comparative scoring system, while an absolute one would be best suited for this specific research.

0.2 Large Language Models

Large Language Models (LLMs) are neural network models trained to process and generate natural language by learning statistical patterns over large text corpora.

LLMs are the culmination of decades of progress in natural language processing (NLP) and machine learning research, and their development is largely responsible for the explosion of artificial intelligence advancements across the late 2010s and 2020s. Popular LLMs have become household names, bringing generative AI to the forefront of the public interest. LLMs are also used widely in enterprises, with organizations investing heavily across numerous business functions and use cases [?].

LLMs are built on a type of neural network architecture called a transformer [?], which employs self-attention mechanisms to model dependencies among tokens in a sequence. Formally, given an input sequence $x_1, \dots, x_n$, the model estimates the conditional distribution

$$P(x_{n+1} \mid x_1, \dots, x_n),$$

and text generation proceeds autoregressively by repeatedly sampling from this distribution.

Training is typically divided into a large-scale pretraining phase followed by alignment stages. During pretraining, the model learns general linguistic and semantic representations through self-supervised objectives. Alignment techniques, such as supervised fine-tuning and reinforcement learning from feedback, adapt the model to follow instructions, respect constraints, and produce outputs that better match user expectations [?].

To better explain the concept, we can say LLMs are, at their core, autoregressive statistical models that generate text by iteratively predicting the most probable subsequent token in a sequence, based on patterns learned from their training data.

0.2.1 Inference and Decoding

At inference time, the model produces logits l_k over the vocabulary, which are transformed into probabilities using a softmax function [?]. Decoding strategies determine how the next token is selected from this distribution.

Modern LLMs typically generate text in a left-to-right, token-by-token fashion. For each prefix, the model computes a probability distribution of the next token over a fixed vocabulary. A decoding method defines how the generated token sequence is derived from these probability estimations. Deterministic approaches, select the most probable token at each step, while stochastic approaches introduce controlled randomness [?].

Some notable deterministic approaches are: “Greedy Search”, which is limited to selecting the token with highest probability at each time step, or “Beam Search” [?], which maintains a beam of the k most probable sequences at each time step, where the hyperparameter k is referred to as the beam width.

Some notable stochastic methods are: Temperature Sampling samples tokens from the estimated next-token distributions. The skewness of distributions can be controlled using a temperature hyperparameter τ , or Top-p Sampling, which only considers the minimal set of most probable tokens that cover a specified percentage p of the distribution.

A central practical constraint is the context window, which limits the total number of tokens that can be processed jointly as input and output. This constraint directly influences prompt design, the amount of contextual information that can be provided, and the feasibility of maintaining conversational state.

0.2.2 Prompting and Interaction

LLMs are typically controlled through prompting, where task instructions and contextual data are encoded in the input.

Prompt engineering has emerged as an indispensable technique for extending the capabilities of LLMs. This approach leverages task-specific instructions, known as prompts, to enhance model efficacy without modifying the core model parameters. Rather than updating the model parameters, prompts allow seamless integration of pre-trained models into downstream tasks by eliciting desired model behaviors solely based on the given prompt.

Prompts can be natural language instructions that provide context to guide the model or learned vector representations that activate relevant knowledge. This rapidly growing field has enabled success across various applications, from question-answering to common sense reasoning [?].

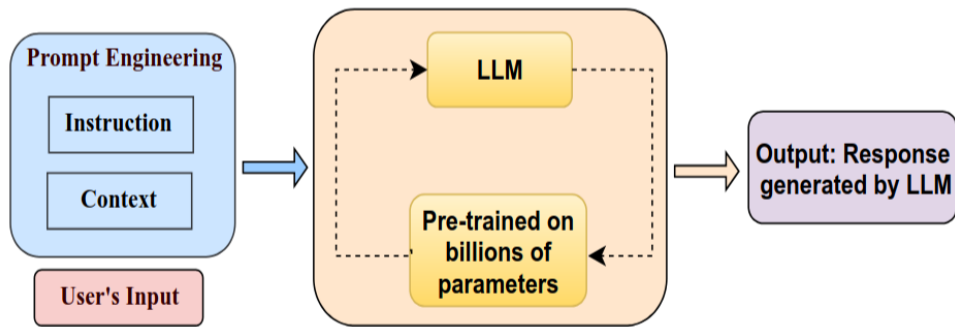


Figure 3: Visual breakdown of prompt engineering components: LLMs trained on extensive data, instruction and context as pivotal elements shaping the prompt, and a user input interface (image was taken from [?]).

Some notable prompt engineering techniques are: “Zero-Shot Prompting” which offers a paradigm shift in leveraging large LLMs. This technique removes the need for extensive training data, instead relying on carefully crafted prompts that guide the model toward novel tasks, or “Few-Shot Prompting” which provides models with a few input-output examples to induce an

understanding of a given task, unlike zero-shot prompting, where no examples are supplied [?].

Interaction may occur in a single-request mode, in which each task is processed independently, or in a multi-turn mode, where previous exchanges remain within the context window.

Prompt structure plays a significant role in model performance, particularly in technical tasks involving structured inputs, strict output formats, or long problem descriptions. The representation of information affects both the interpretability of the prompt and the effective use of the context window.

0.2.3 Application Programming Interfaces

In practical settings, LLMs are accessed through Application Programming Interfaces (APIs). An application programming interface, or API, enables commercial, military or private entities to make the data and functionality of their applications or systems available to external third-party developers, commercial partners, and internal departments within their own organizations. The usage of a defined interface enables services and products to interact with one another and benefit from each other’s information and features. This interface is manipulated and programmed by developers to interact with other software, services or systems; they are not required to understand how a specific API is developed, and neither are the software’s end-users. In short, an API is a contract between pieces of applications serving the main software once integrated into the source code of the main application [?].

An LLM API provides a standardized mechanism for submitting input messages and receiving model outputs, together with configurable inference parameters such as temperature. API-based access introduces operational constraints, including limits on the number of requests or tokens processed within a given time interval, as well as maximum context sizes per interaction. But enables the use of complex LLMs without the need of high computational power.

0.2.4 Relevance to Solver Selection

In this research, the aim is to test the potential of LLMs when used for solver selection. Where the prompt is engineered in order to help the LLM evaluate all the characteristics of a problem, to make a decision that is as functional as possible.

When applied to solver selection, an LLM functions as a decision component that maps problem representations to solver choices. Its performance depends on both theoretical aspects, such as its ability to model complex input-output relationships, and practical factors, including prompt formulation, context management, and API-level parameters.